

2020 시스템 프로그래밍
- Malloc Lab -

제출일자	2020.12.14
분 반	02
이 름	최재범
학 번	201702081

Naive

```
Results for mm malloc:
  valid  util   ops   secs   Kops  trace
yes     94%    10 0.000000 62030 ./traces/malloc.rep
yes     77%    17 0.000000 77978 ./traces/malloc-free.rep
yes    100%    15 0.000000 50161 ./traces/corners.rep
* yes    71%  1494 0.000018 81880 ./traces/perl.rep
* yes    68%   118 0.000002 62536 ./traces/hostname.rep
* yes    65% 11913 0.000122 97854 ./traces/xterm.rep
* yes    23%   5694 0.000065 87618 ./traces/amptjp-bal.rep
* yes    19%   5848 0.000066 89035 ./traces/cccp-bal.rep
* yes    30%   6648 0.000074 89252 ./traces/cp-decl-bal.rep
* yes    40%   5380 0.000064 83930 ./traces/expr-bal.rep
* yes     0%  14400 0.000193 74804 ./traces/coalescing-bal.rep
* yes    38%   4800 0.000072 66222 ./traces/random-bal.rep
* yes    55%   6000 0.000050 120940 ./traces/binary-bal.rep
10     41%  62295 0.000726 85838

Perf index = 26 (util) + 40 (thru) = 66/100
```

매크로 및 함수 설명

```
/* single word (4) or double word (8) alignment */
#define ALIGNMENT 8

/* rounds up to the nearest multiple of ALIGNMENT */
#define ALIGN(size) (((size) + (ALIGNMENT-1)) & ~0x7)

#define SIZE_T_SIZE (ALIGN(sizeof(size_t)))

#define SIZE_PTR(p) ((size_t*)((char*)(p)) - SIZE_T_SIZE)
```

ALIGNMENT : 블록 단위 명시 (8바이트)

ALIGN(size) : 블록 단위인 8의 배수로 근사함(올림)

SIZE_T_SIZE : size_t를 주소 크기인 8의 배수로 올림

SIZE_PTR(p) : 헤더에 있는 블록의 크기를 얻음

```
void *malloc(size_t size)
{
    int newsize = ALIGN(size + SIZE_T_SIZE);
    unsigned char *p = mem_sbrk(newsize);
    //dbg_printf("malloc %u => %p\n", size, p);

    if ((long)p < 0)
        return NULL;
    else {
        p += SIZE_T_SIZE;
        *SIZE_PTR(p) = size;
        return p;
    }
}
```

malloc() : size를 헤더의 크기를 고려하여 8의 배수로 근사, mem_sbrk() 이용하여 공간을 할당받고 크기정보를 포함한 헤더를 블록에 추가함.

```

void *realloc(void *oldptr, size_t size)
{
    size_t oldsize;
    void *newptr;

    /* If size == 0 then this is just free, and we return NULL. */
    if(size == 0) {
        free(oldptr);
        return 0;
    }

    /* If oldptr is NULL, then this is just malloc. */
    if(oldptr == NULL) {
        return malloc(size);
    }

    newptr = malloc(size);

    /* If realloc() fails the original block is left untouched */
    if(!newptr) {
        return 0;
    }

    /* Copy the old data. */
    oldsize = *SIZE_PTR(oldptr);
    if(size < oldsize) oldsize = size;
    memcpy(newptr, oldptr, oldsize);

    /* Free the old block. */
    free(oldptr);

    return newptr;
}

```

realloc() : 새로운 크기와 기존 포인터를 받아서, 예전에 할당받은 내용을 가능한 만큼 새로 할당받은 블록에 옮김. 이후 기존 공간을 free함

```

/*
 * mem_sbrk - simple model of the sbrk function. Extends the heap
 * by incr bytes and returns the start address of the new area. In
 * this model, the heap cannot be shrunk.
 */
void *mem_sbrk(int incr)
{
    char *old_brk = mem_brk;

    if ( (incr < 0) || ((mem_brk + incr) > mem_max_addr)) {
        errno = ENOMEM;
        fprintf(stderr, "ERROR: mem_sbrk failed. Ran out of memory...\n");
        return (void *)-1;
    }
    mem_brk += incr;
    return (void *)old_brk;
}

```

mem_sbrk : heap 영역에 새로운 메모리 추가

Implicit

```
Results for mm malloc:
  valid  util   ops    secs    Kops  trace
  yes    34%    10    0.000000 22461 ./traces/malloc.rep
  yes    28%    17    0.000001 33093 ./traces/malloc-free.rep
  yes    96%    15    0.000001 20930 ./traces/corners.rep
* yes    86%   1494  0.002283   654 ./traces/perl.rep
* yes    75%   118  0.000023   5122 ./traces/hostname.rep
* yes    91%  11913  0.121260    98 ./traces/xterm.rep
* yes    99%   5694  0.011600   491 ./traces/amptjp-bal.rep
* yes    99%   5848  0.010774   543 ./traces/cccp-bal.rep
* yes    99%   6648  0.017518   379 ./traces/cp-decl-bal.rep
* yes   100%   5380  0.012954   415 ./traces/expr-bal.rep
* yes    66%  14400  0.000328 43850 ./traces/coalescing-bal.rep
* yes    93%   4800  0.009932   483 ./traces/random-bal.rep
* yes    55%   6000  0.029429   204 ./traces/binary-bal.rep
10      86%  62295  0.216101   288

Perf index = 56 (util) + 12 (thru) = 67/100
```

구현 방법

```
// 매크로
#define WSIZE      4           // Word 크기
#define DSIZE      8           // Double Word 크기
#define CHUNKSIZE  (1 << 12)  // 초기 heap 크기
#define OVERHEAD    8         // Header + Footer 크기
#define MAX(x, y)   ((x) > (y) ? (x) : (y)) // 둘중 더 큰 값

#define PACK(size, alloc) ((size) | (alloc)) // size, alloc 값을 하나의 word로 묶음
#define GET(p)            (*(unsigned int *) (p)) // 포인터 p가 가리키는 위치에서 word 크기의 값을 읽음
#define PUT(p, val)        (*(unsigned int *) (p)) = (val) // 포인터 p가 가리키는 위치에 word 크기의 val 값을 씀

#define GET_SIZE(p) (GET(p) & ~0x7) // Header에서 block size 읽을 때 사용 (한 워드 읽고 하위3비트 버림)
#define GET_ALLOC(p) (GET(p) & 0x1) // Header에서 alloc 여부 읽을 때 사용 (한 워드 읽고 하위1비트 읽음)
#define HDRP(bp) ((char *) (bp) - WSIZE) // Header 주소 계산
#define FTRP(bp) ((char *) (bp) + GET_SIZE(HDRP(bp)) - DSIZE) // Header로부터 크기 참조하여, Footer 주소 계산
#define NEXT_BLKP(bp) ((char *) (bp) + GET_SIZE(((char *) (bp) - WSIZE))) // 다음 block의 주소 계산
#define PREV_BLKP(bp) ((char *) (bp) - GET_SIZE(((char *) (bp) - DSIZE))) // 이전 block의 주소 계산
```

매크로

```
// Initialize: return -1 on error, 0 on success.
int mm_init(void) {
    if( (heap_listp = mem_sbrk(4 * WSIZE)) == NULL ) // 초기 Empty Heap 생성
        return -1; // heap_listp = 새로 생성되는 heap 영역의 시작 주소

    PUT(heap_listp, 0); // 정렬을 위해서 의미없는 값 삽입
    PUT(heap_listp + WSIZE, PACK(OVERHEAD, 1)); // Prologue Header
    PUT(heap_listp + DSIZE, PACK(OVERHEAD, 1)); // Prologue Footer
    PUT(heap_listp + WSIZE + DSIZE, PACK(0, 1)); // Epilogue Header
    heap_listp += DSIZE;

    if( (extend_heap(CHUNKSIZE / WSIZE)) == NULL ) // CHUNKSIZE 바이트의 Free Block만큼 Empty Heap 확장
        return -1; // 생성된 Empty Heap을 Free Block으로 확장
    // WSIZE로 align 되어있지 않으면 에러
    return 0;
}
```

mm_init : 초기 empty heap을 생성하고, prologue header/footer, epilogue header를 집어넣음. 이후 extend_heap(mem_sbrk 사용)을 사용하여 힙을 확장함.

```

// extend_heap
static void *extend_heap(size_t words)
{
    char *bp;
    size_t size;

    // Allocate an even number of words to maintain alignment
    size = (words % 2) ? (words + 1) * WSIZE : words * WSIZE;
    if( (long)(bp = mem_sbrk(size)) == -1 )
        return NULL;

    // Free Block의 Header, Footer, Epilogue Header 넣기
    PUT(HDRP(bp), PACK(size, 0)); // Header
    PUT(FTRP(bp), PACK(size, 0)); // Footer
    PUT(HDRP(NEXT_BLKP(bp)), PACK(0, 1)); // Epilogue Header

    // 이전 Block이 Free라면 병합
    return coalesce(bp);
}

```

extend_heap : word 개수를 2의 배수로 만들고, 총 크기만큼 mem_sbrk로 Free block 할당받음. 이후 Header / Footer, Epilogue Header 넣고 병합을 위해 coalesce 호출함.

```

// coalesce : 병합
static void *coalesce(void *bp) {
    size_t prev_alloc = GET_ALLOC(FTRP(PREV_BLKP(bp))); // 이전 블록의 할당 여부
    size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKP(bp))); // 다음 블록의 할당 여부
    size_t size = GET_SIZE(HDRP(bp)); // 현재 블록의 크기

    // Case 1 : 이전, 다음 블록의 최하위 bit가 둘다 1 (할당됨) : 병합 없이 bp return
    if(prev_alloc && next_alloc)
        return bp;

    // Case 2 : 이전 블록의 최하위 bit가 1이고 (할당됨), 다음 블록의 최하위 bit가 0임 (미할당) : 다음 블록과 병합 후 bp return
    else if(prev_alloc && !next_alloc) {
        size += GET_SIZE(HDRP(NEXT_BLKP(bp))); // 다음 블록의 헤더로부터, 크기 읽어옴
        PUT(HDRP(bp), PACK(size, 0)); // bp의 Header에 block size와 alloc = 0을 저장
        PUT(FTRP(bp), PACK(size, 0)); // bp의 Footer에 block size와 alloc = 0을 저장
    }

    // Case 3 : 이전 블록의 최하위 bit가 0이고 (미할당), 다음 블록의 최하위 bit가 1임 (할당됨) :
    // 이전 블록과 병합 후 새로운 bp return
    else if(!prev_alloc && next_alloc) {
        size += GET_SIZE(HDRP(PREV_BLKP(bp))); // 이전 블록의 헤더로부터, 크기 읽어옴
        PUT(FTRP(bp), PACK(size, 0)); // 현재 블록의 Footer에 block size와 alloc = 0 저장
        PUT(HDRP(PREV_BLKP(bp)), PACK(size, 0)); // 이전 블록의 Header에 block size와 alloc = 0 저장
        bp = PREV_BLKP(bp); // 이전 블록을 새로운 블록으로 설정
    }

    // Case 4 : 이전, 다음 블록의 최하위 bit가 둘다 0 (미할당) :
    // 이전, 다음 블록을 모두 병합한 뒤 새로운 bp return
    else {
        size += GET_SIZE(HDRP(PREV_BLKP(bp))) + GET_SIZE(FTRP(NEXT_BLKP(bp))); // 전,후,현재 크기 더함
        PUT(HDRP(PREV_BLKP(bp)), PACK(size, 0)); // 이전 블록의 Header에 block size와 alloc = 0 저장
        PUT(FTRP(NEXT_BLKP(bp)), PACK(size, 0)); // 다음 블록의 Footer에 block size와 alloc = 0 저장
        bp = PREV_BLKP(bp); // 이전 블록을 새로운 블록으로 설정
    }

    return bp; // 병합된 블록의 주소 bp return
}

```

coalesce : 헤더와 푸터의 alloc 비트를 참조하여 Free 여부를 판단하며, 이전/다음 블록이 Free이면 통합하여 하나의 Free block으로 만듦. 앞/뒤에 블록이 존재하는 4가지 경우 따라 다르게 처리를 하며, 이전 블록이 통합되는 경우 이전 블록의 포인터가 bp가 되도록 갱신함.


```

// malloc
void *malloc (size_t size) {
    size_t asize;
    size_t extendsize;
    char *bp;

    if(size == 0)
        return NULL;
    if(size <= DSIZE)
        asize = 2 * DSIZE;
    else
        asize = DSIZE * ((size + (DSIZE) + (DSIZE-1)) / DSIZE);

    // Free 공간 찾기
    if ( (bp = find_fit(asize)) != NULL ) {
        place(bp, asize);
        return bp;
    }

    // 못찾음, 메모리 더 얻어서 재시도
    extendsize = MAX(asize, CHUNKSIZE);
    if( (bp = extend_heap(extendsize / WSIZE)) == NULL )
        return NULL;
    place(bp, asize);
    return bp;
}

```

malloc : 8바이트 이하의 요청에 대해서는 최소 블록 사이즈인 16바이트를 할당하고, 그 이상에 대해서는 인접한 8의 배수로 반올림한다. 이후 find_fit 함수를 호출하여 힙을 탐색하며, 찾은 free block(최초할당)에 place 함수를 적용하여 alloc 표시를 한다.

```

// First-fit 탐색
static void *find_fit(size_t asize)
{
    // First-fit 탐색
    void *bp;
    for(bp = heap_listp; GET_SIZE(HDRP(bp)) > 0; bp = NEXT_BLKp(bp)) {
        // ALLOC 비트가 0이면서, 해당 블록의 사이즈가 요청된 사이즈 이상이어야 함
        if( !GET_ALLOC(HDRP(bp)) && (asize <= GET_SIZE(HDRP(bp))) )
            return bp;
    }
    return NULL;    // No Fit
}

```

find_fit : 헤더의 alloc 비트가 0이면서, 요구하는 사이즈 이상의 블록을 최초로 만날 시 바로 반환한다. (최초할당)

```

// Place : find-fit으로 찾아낸 free block의 Header, Footer alloc 비트를 1로 설정
static void place(void *bp, size_t asize)
{
    size_t csize = GET_SIZE(HDRP(bp));

    // 찾아낸 Free Block이 요청된 크기보다 16이상 큰 경우 : 사용할 만큼만 alloc 1 세팅
    if((csize - asize) >= (2*DSIZE)) {
        PUT(HDRP(bp), PACK(asize, 1));
        PUT(FTRP(bp), PACK(asize, 1));
        bp = NEXT_BLKp(bp);
        PUT(HDRP(bp), PACK(csize - asize, 0));
        PUT(FTRP(bp), PACK(csize - asize, 0));
    }
    else {
        PUT(HDRP(bp), PACK(csize, 1));
        PUT(FTRP(bp), PACK(csize, 1));
    }
}

```

place : 찾은 블록에 alloc 비트를 세팅하여 할당되었음을 표시, 블록 전체 크기와 할당함. 크기가 최소 블록크기인 16바이트 이상 차이난다면 쪼개서 일부만 할당.

```

// free
void free (void *bp) {
    if (bp == 0) return;           // 잘못된 Free 요청인 경우 이전 프로시저로 return
    size_t size = GET_SIZE(HDRP(bp)); // bp의 헤더에서 block size를 읽어옴

    // Header와 Footer의 최하위 1bit (Allocation bit) 만 수정 ~ 실제 데이터는 안지움
    PUT(HDRP(bp), PACK(size, 0));   // bp의 Header에 block size와 alloc = 0을 저장
    PUT(FTRP(bp), PACK(size, 0));   // bp의 Footer에 block size와 alloc = 0을 저장

    coalesce(bp);                  // 주위에 빈 블록이 있을 시 병합
}

```

free : 블록 포인터의 헤더로부터 블록 사이즈를 읽어온 뒤, 헤더와 푸터의 alloc 비트에 0을 세팅하여 비할당 표시를 하고, 빈 블록들을 coalesce를 통해 결합한다.

realloc : naive 것을 그대로 붙여넣음.